

GCP Storage Options Handbook

Cloud Storage, Cloud SQL, Spanner, Firestore & Bigtable — Architecture, Costing, and When to Use Which

Google Cloud offers five fundamentally different storage services, each solving a different shape of problem. Picking the wrong one isn't just suboptimal — a relational workload forced into Firestore, or a high-throughput time-series workload forced into Cloud SQL, will hit a wall that no amount of tuning fixes. This handbook explains what each service actually is, how it works under the hood, what it costs, and gives a clear decision framework for choosing between them — plus how they connect together in a real, multi-service architecture.

The one-sentence version of this whole handbook: Cloud Storage for files, Cloud SQL for straightforward relational apps in one region, Spanner for relational data that must scale globally with strong consistency, Firestore for mobile/web apps needing real-time sync with a flexible schema, and Bigtable for massive-scale, high-throughput, single-key-lookup workloads like IoT and time series.

1. The Decision Framework, At a Glance

Before the deep dives, here is the single most useful artifact in this handbook — the at-a-glance comparison worth returning to every time a new workload needs a home.

	Cloud Storage	Cloud SQL	Spanner	Firestore	Bigtable
Data model	Object/blob	Relational (rows/tables)	Relational, globally distributed	Document/NoSQL	Wide-column NoSQL
Query language	None (get/put by key)	Full SQL	Full SQL (+ strong consistency)	Document queries, limited joins	Row-key lookup, HBase API
Consistency	Strong (per-object)	Strong (single region)	Strong, globally (via TrueTime)	Strong (multi-region)	Eventually/strong per-row, no cross-row transactions at scale
Scale ceiling	Effectively unlimited (exabytes)	Vertical scaling + read replicas; TBs	Horizontal, PBs, global	Very large, auto-scaling	PBs, massive read/write throughput
Typical latency	Tens-hundreds of ms	Single-digit ms (same region)	Single-digit ms, low tens ms cross-region	Single-digit to tens of ms	Single-digit ms at massive scale
Best for	Files, backups, data lake, ML training data	Traditional relational apps, moderate scale	Global-scale relational systems (finance, inventory)	Mobile/web apps, real-time sync, flexible schema	IoT, time series, ad-tech, huge key-value/wide-column workloads
Pricing unit	\$/GB stored (by class) + operations + egress	\$/vCPU-hour + \$/GB storage + network	\$/processing unit or node-hour + \$/GB storage	\$/document read, write, delete + \$/GB storage	\$/node-hour + \$/GB storage (SSD/HDD)

2. Cloud Storage — Object Storage

What it is: a flat namespace of immutable objects (files) inside buckets, accessed by key (the object name) — not a database at all, but the foundation everything else in a data platform sits on top of (raw file landing zones, ML training data, backups, static web assets).

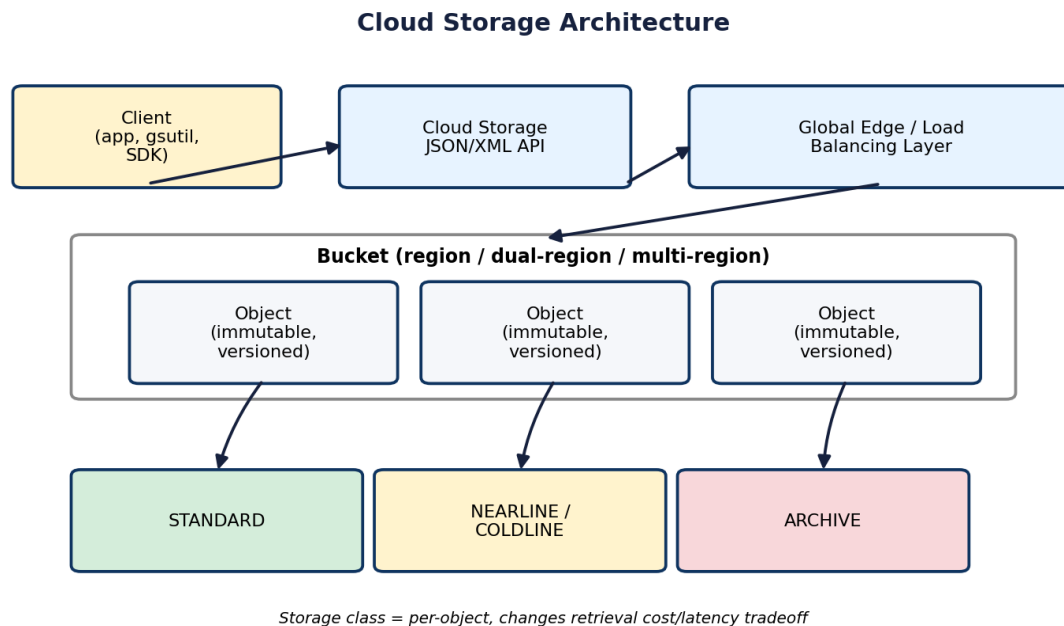


Figure 1: Objects live in a bucket bound to one region/dual-region/multi-region. Storage class is set per-object and trades retrieval cost/latency for storage price.

2.1 How It Works

Every object is versioned and immutable — an "update" is actually a new object generation replacing the old one (Day 1's GCS lab covers this directly). Buckets are bound to a location (single region, dual-region, or multi-region) at creation and cannot be moved without copying data to a new bucket. Consistency is strong for both reads and writes — a successful upload is immediately visible to every subsequent read, no eventual-consistency window to reason about.

2.2 Costing

Storage: priced per GB per month, varying by class — Standard is the most expensive to store but free to retrieve; Archive is the cheapest to store but has both a retrieval fee and a minimum storage duration (365 days) that applies even if you delete early.

Operations: Class A operations (writes, listing) cost more per-operation than Class B (reads) — a high-frequency-write workload costs meaningfully more than a write-once-read-many one at the same total byte volume.

Network egress: free within the same region to other GCP services; charged when data leaves GCP entirely or crosses regions — a common, easy-to-miss line item on a cost review.

2.3 When to Use / When Not To

Use for: files that don't fit a row/column shape (images, PDFs, model checkpoints), a landing zone before loading into BigQuery, or Iceberg/Parquet lakes queried directly via BigLake. **Don't use for:** anything needing per-record query/filter/transaction semantics — that's every other service in this handbook.

3. Cloud SQL — Managed Relational Databases

What it is: a fully managed MySQL, PostgreSQL, or SQL Server instance — the same relational engine you already know, minus the operational burden of patching, backups, and failover.

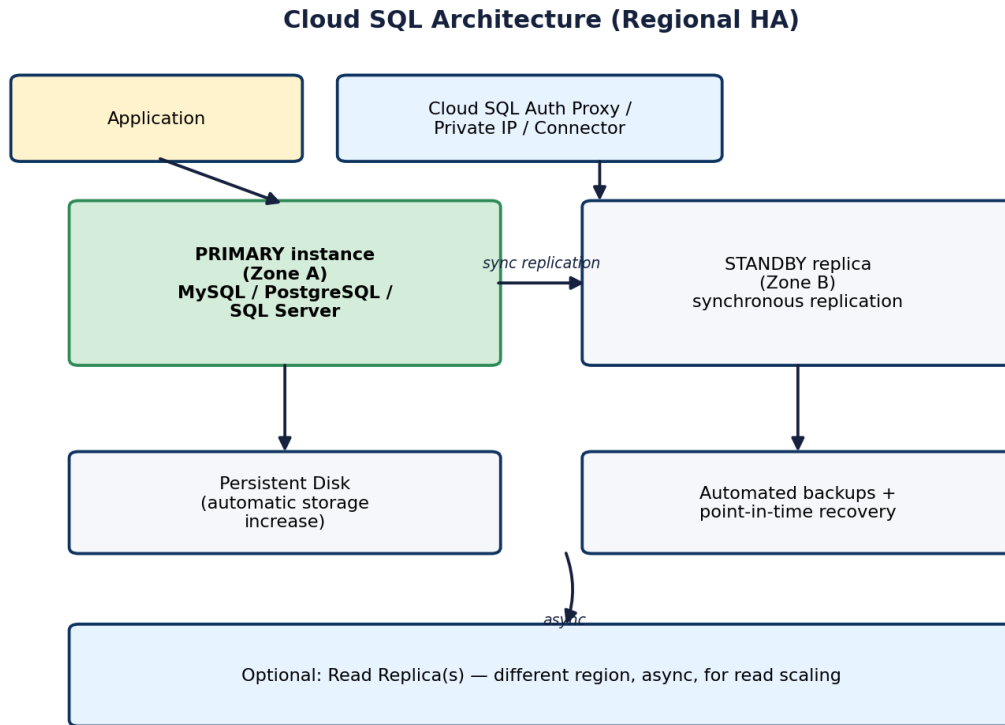


Figure 2: A primary instance replicates synchronously to a standby in a second zone for automatic failover; optional async read replicas (even cross-region) scale read traffic.

3.1 How It Works

Regional HA configuration keeps a synchronous standby in a second zone — if the primary zone fails, Cloud SQL fails over automatically, typically within a minute or so. Storage auto-increases as data grows (no manual resizing), and automated backups plus point-in-time recovery (via write-ahead log replay) protect against logical mistakes, not just hardware failure. Applications connect via the **Cloud SQL Auth Proxy** or a private IP — never expose a public IP with password-only auth in production.

3.2 Costing

Instance: priced per vCPU and GB of memory per hour — you choose a fixed machine shape (or use the newer Enterprise Plus editions' more flexible sizing); billed whether or not the instance is actively serving queries, unlike BigQuery's on-demand model.

Storage & backups: billed per GB for both the primary data disk and backup storage — keeping many long-retained backups is a real, recurring cost worth reviewing periodically.

Network: egress to the internet is charged; traffic to other GCP services in the same region over private IP is not.

3.3 When to Use / When Not To

Use for: a straightforward transactional app (orders, users, inventory) that fits comfortably in one region and doesn't need Spanner's global scale. **Don't use for:** a workload that will outgrow a single instance's vertical scaling ceiling, or that needs strong consistency across multiple regions — that's Spanner's job.

4. Cloud Spanner — Globally Distributed Relational Database

What it is: a relational database (full SQL, transactions, joins) that scales horizontally like a NoSQL system and stays strongly consistent even when replicated across continents — a combination no traditional relational database can offer without heavy manual sharding.

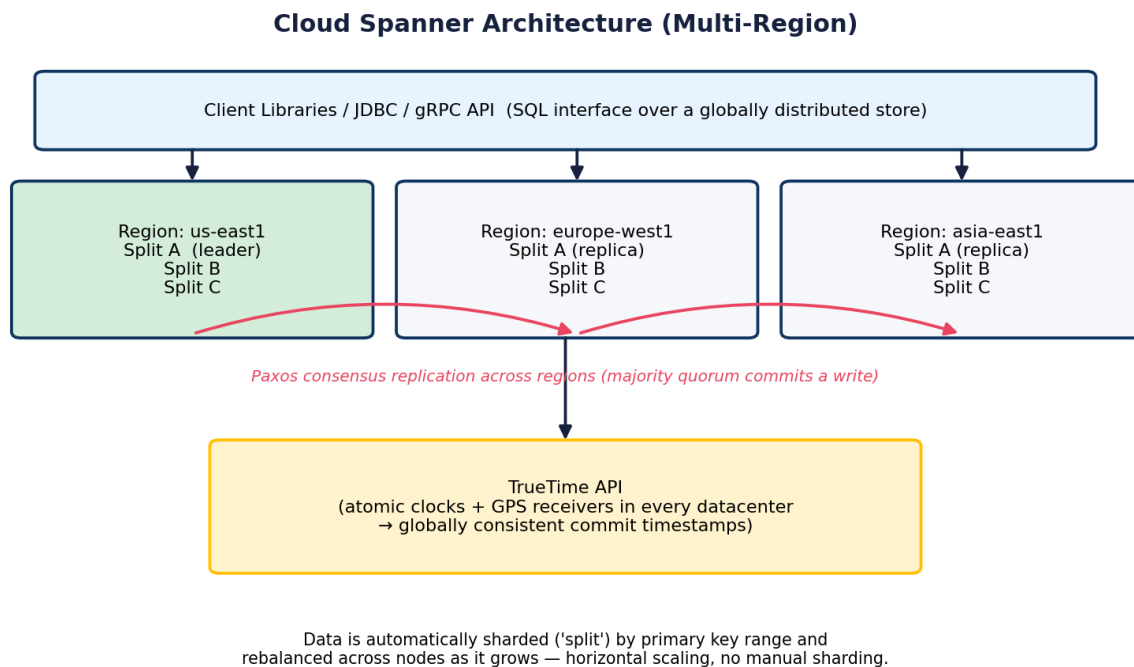


Figure 3: Data is automatically split by primary-key range and replicated across regions via Paxos consensus; TrueTime gives every commit a globally meaningful timestamp.

4.1 How It Works

Spanner automatically breaks tables into **splits** (contiguous key ranges) and distributes them across machines — as data grows, Spanner rebalances splits without any manual sharding logic in application code. Each split is replicated across regions using the **Paxos** consensus protocol; a write commits once a majority of replicas acknowledge it. The genuinely novel piece is **TrueTime** — Google's globally synchronized clock (atomic clocks + GPS receivers in every datacenter) that lets Spanner assign a commit timestamp with a bounded uncertainty window small enough to guarantee external consistency globally, something no other distributed relational database can claim.

4.2 Costing

Compute: priced per **processing unit** or node-hour (1,000 processing units = 1 node); the entry-level offering (a fraction of a node, in processing units) makes small workloads affordable, while production systems typically run multiple full nodes per region for both throughput and availability.

Storage: priced per GB per month, replicated across however many regions your instance config specifies — a 3-region multi-region config stores (and pays for) 3x the raw data.

Network: cross-region replication traffic and external egress are both billed — the global consistency guarantee has a real network cost underpinning it.

4.3 When to Use / When Not To

Use for: financial ledgers, global inventory systems, or any relational workload that must remain strongly consistent while serving users on multiple continents. **Don't use for:** a single-region app with modest scale — Spanner's minimum cost and operational complexity aren't justified until you actually need its global consistency guarantee.

5. Firestore — Document Database

What it is: a NoSQL document database built for mobile and web apps — flexible schema (documents in the same collection can have different fields), real-time synchronization to connected clients, and offline support built into the client SDKs.

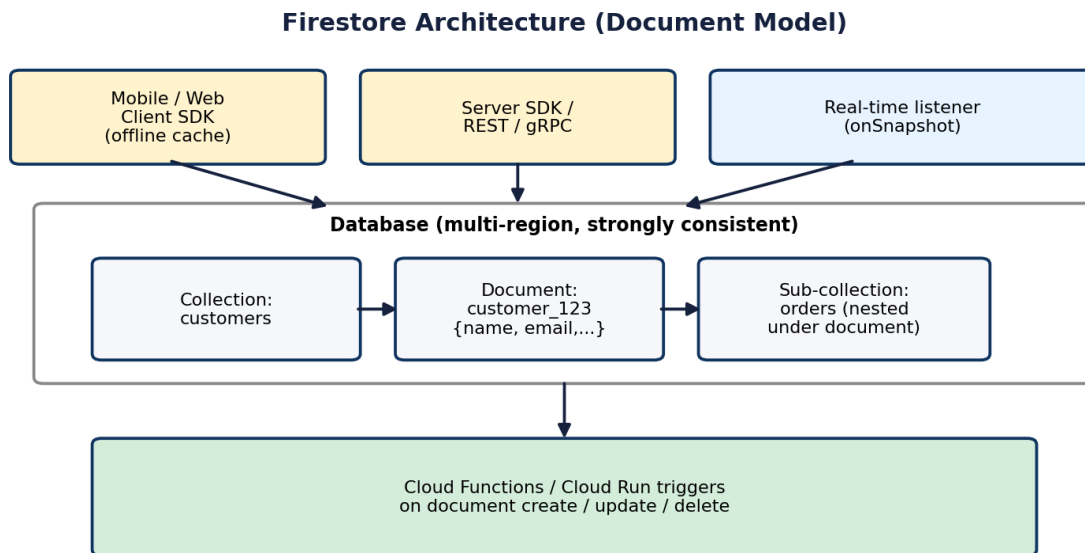


Figure 4: Documents live in collections and can have nested sub-collections; client SDKs maintain a live, synced connection via real-time listeners, with offline caching built in.

5.1 How It Works

Data is organized as **documents** (JSON-like objects) grouped into **collections**; documents can have **sub-collections** nested underneath them (e.g. a customer document with an orders sub-collection). Firestore is strongly consistent even across multi-region deployments — a write is immediately visible to subsequent reads, not eventually. Client SDKs (iOS, Android, Web) can attach a **real-time listener** to a query; Firestore then pushes any matching change to every connected client automatically, without polling — this is the core feature that makes Firestore the default choice for chat apps, live dashboards, and collaborative tools.

5.2 Costing

Per-operation pricing: billed per **document read, write, and delete** — not per GB scanned like BigQuery, and not per instance-hour like Cloud SQL/Spanner. A query that matches 500 documents bills as 500 reads, regardless of how much data each document contains, which makes query design (fetching only what you need) a direct cost lever.

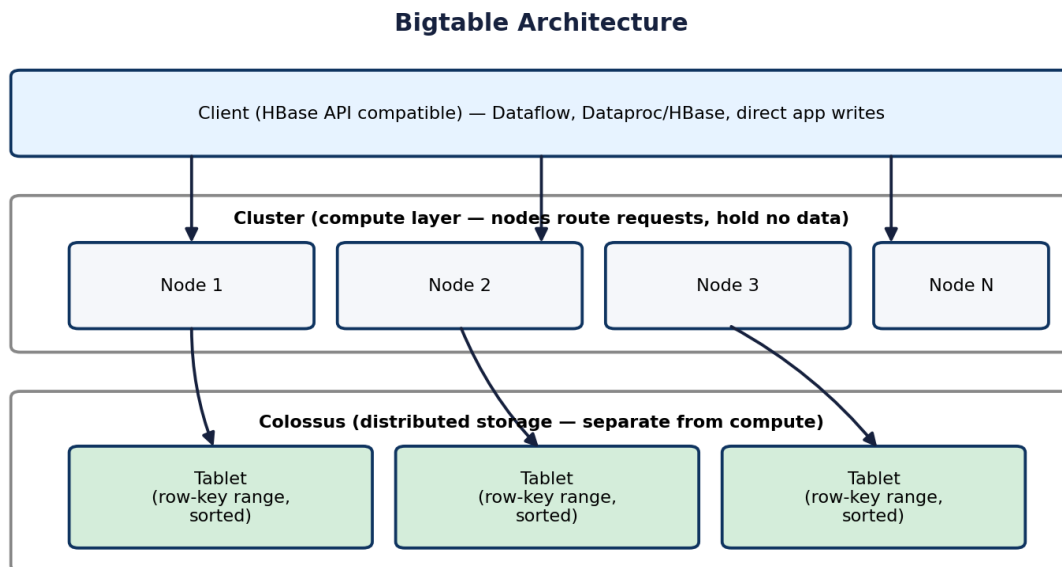
Storage & network: storage is priced per GB per month; network egress to the internet is billed, same pattern as the other services.

5.3 When to Use / When Not To

Use for: mobile/web apps needing real-time UI updates, flexible/evolving schemas, and offline support. **Don't use for:** workloads needing complex multi-table joins or ACID transactions across a large working set — Firestore's transaction model is scoped and not a substitute for genuine relational joins at scale.

6. Bigtable — Wide-Column NoSQL at Massive Scale

What it is: the same underlying system Google uses internally for Search, Maps, and Gmail — a sparse, distributed, sorted map built for massive throughput and low-latency single-row lookups, exposed externally via an HBase-compatible API.



Row key design determines performance: rows are sorted lexicographically by key, so good key design (avoiding hotspotting) is the single biggest factor in Bigtable performance.

Figure 5: Compute (nodes, in a cluster) and storage (Colossus, holding tablets) are separate layers — you can resize compute without moving data, and vice versa.

6.1 How It Works

Rows are sorted lexicographically by **row key** and split into **tablets** — contiguous key ranges — distributed across **nodes** in a cluster. Critically, nodes are compute-only; the actual data lives in **Colossus** (Google's distributed file system), decoupled from the compute layer — this is why resizing a Bigtable cluster (adding/removing nodes) takes effect in minutes rather than requiring a data migration. **Row key design is the single biggest performance lever:** since rows are sorted by key, a poorly chosen key (e.g. a sequential timestamp as the leading key component) concentrates all writes onto one tablet — a hotspot — while a well-designed key (e.g. a reversed timestamp, or a hash prefix) spreads load evenly.

6.2 Costing

Nodes: priced per node-hour — unlike Firestore's per-operation model, Bigtable bills for provisioned capacity whether or not it's fully utilized, so right-sizing cluster node count to actual throughput needs is the primary cost lever.

Storage: priced per GB per month, with SSD storage costing more than HDD — SSD is the default and right choice for nearly all latency-sensitive workloads; HDD suits only very large, latency-tolerant archives.

6.3 When to Use / When Not To

Use for: IoT sensor data, time-series metrics, ad-tech bidding logs, financial market data — anything with massive write throughput and simple key-based lookup patterns. **Don't use for:** anything needing SQL joins, multi-row ACID transactions, or ad-hoc analytical queries across arbitrary columns — Bigtable rewards workloads that know their access pattern in advance (via row key design) and punishes those that don't.

7. How These Services Interact in a Real Architecture

These five services are rarely used in isolation — a real platform typically combines several, each handling the part of the workload it's actually good at.

How These Services Typically Interact in One Architecture

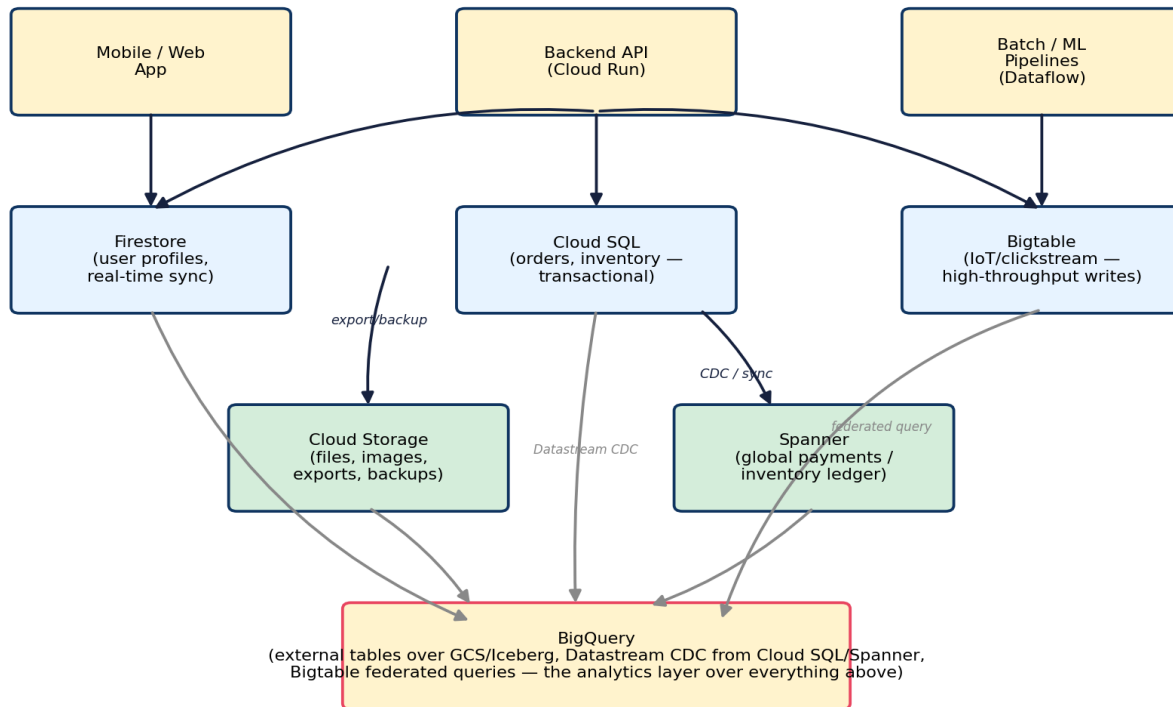


Figure 6: A typical multi-service architecture — operational stores feed an analytics layer (BigQuery) via CDC, export, or federated query, without duplicating application logic.

Interaction	Mechanism	Why
Cloud SQL / Spanner → BigQuery	Datastream (CDC)	Continuously replicate operational changes into the analytics warehouse without batch ETL jobs (covered in the Module 3 orchestration notebook)
Cloud Storage → BigQuery	External tables / BigLake	Query files in place without a separate load step (covered in the BigQuery Advanced notebook)
Firestore → BigQuery	Scheduled export + load, or Firestore's native BigQuery export	Analyze document data with SQL without querying production Firestore directly
Bigtable → BigQuery	Federated queries (BigQuery reads Bigtable directly)	Ad-hoc analytical queries over high-throughput operational data, without copying it
Any operational store → Cloud Storage	Automated backups / exports	Durable, cheap long-term retention outside the operational database itself
Dataflow	Reads/writes almost all five	The common data-movement layer stitching these services together in pipelines

8. Decision Tree — Which One Should I Use?

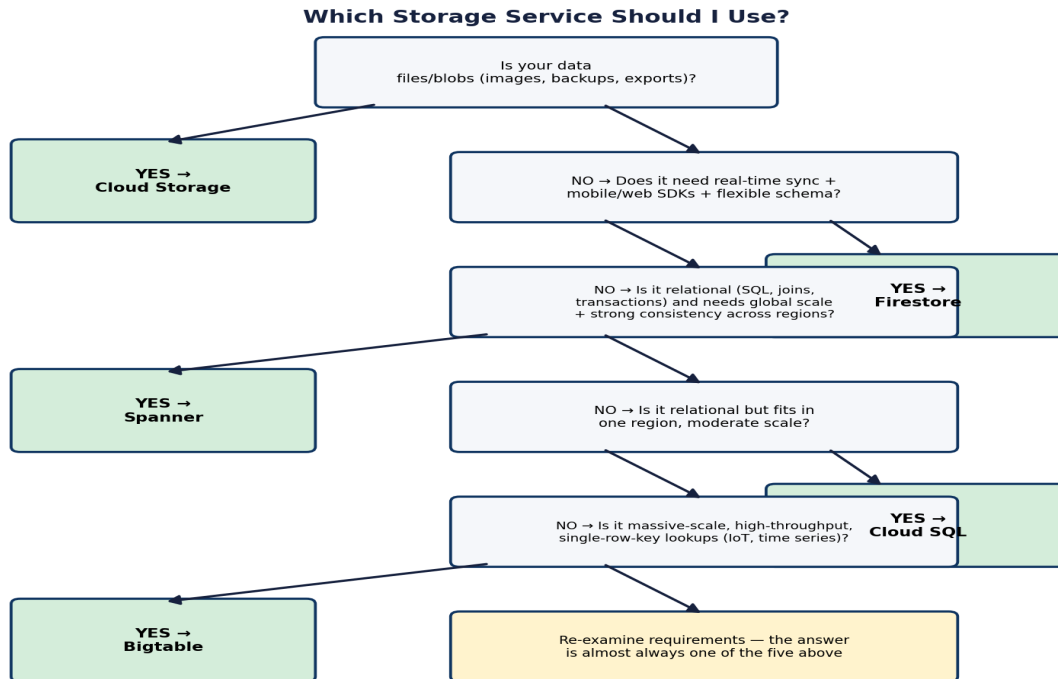


Figure 7: Work top to bottom — the first "yes" you hit is almost always the right answer.

A few edge cases worth calling out explicitly:

- **"Relational, but might need global scale later"** — start with Cloud SQL if the global requirement isn't concrete yet; migrating from Cloud SQL to Spanner later is a real project, but so is over-engineering for global scale you may never need.
- **"Document-shaped data, but only ever queried from a backend, never a mobile client"** — Firestore still works, but also consider whether the data is really relational underneath (in which case Cloud SQL might be the simpler, cheaper choice).
- **"High write throughput, but I also need complex queries"** — this is the case that most often means "more than one service," not "pick harder": write to Bigtable for throughput, federated-query or export into BigQuery for the complex analytical side.

Where this fits with the rest of the course: Day 1's hands-on labs cover Cloud Storage and BigQuery directly; Module 3's orchestration notebooks cover Datastream CDC from an operational database into BigQuery; the BigQuery Advanced notebook covers external tables and BigLake. This handbook is the conceptual map tying all of those hands-on pieces to the other operational stores (Cloud SQL, Spanner, Firestore, Bigtable) that a real architecture would also include.